

Statement Insights

Short code, concrete walk-through — what each statement means when you plug in a real value

October 2026

How each card works. A statement; the sentence you say for it; then a concrete call with a real value so you can see what you actually hold afterwards; then the one review question it raises. The habit to build: *never read a line abstractly; plug in a value and say what the variable holds.*

Async and Promises

1. An async function hands you a wrapper, not the value

```
const f = async (x: number): Promise<number> => { return x * 2 };
```

Say: “**f** takes a number, returns a promise of a number, body returns **x** times 2.”

Plug in 3: the body computes $3 * 2 = 6$. But **f** is **async**, so you do not get the **6** directly. You get a Promise that will contain **6**.

```
const p = f(3);           // p holds Promise<number>: a ticket; the 6 is inside it
const n = await f(3);     // n holds 6: the ticket was cashed
p + 1                     // x adding 1 to a ticket → "[object Promise]1"
n + 1                     // ✓ 7
```

Review question: does every caller of **f** have **await**? One that uses the result directly is holding the ticket.

2. **await** cashes the ticket; without it the variable is the ticket

```
const u = getUser("42");
```

Say: “**u** is set to whatever **getUser** returns.”

Plug in: **getUser** is async, so it returns **Promise<User>**. **u** holds the Promise. **u.name** is **undefined** because a Promise has no **name** key.

```
const u = getUser("42");   // u: Promise<User>
u.name                     // undefined x
const v = await getUser("42"); // v: User
v.name                     // "Ann" ✓
```

Review question: is the function being called **async** (or does it return a Promise)? If yes and there is no **await**, that is the bug.

3. **Promise<T>**: same wrapper, different contents

```
Promise<number>    Promise<User>    Promise<User[]>    Promise<void>
```

Say: “a promise of a number / of a User / of an array of Users / of nothing useful.”

Plug in: after **await**, you hold **6** / **{ id: "42", name: "Ann" }** / **[{...}, {...}]** / **undefined**. The **T** names what comes out; the wrapper is always the same.

Review question: does the code use the result as a **T**, or as a Promise?

4. **Promise.all** turns an array of tickets into one ticket for an array

```
const users = await Promise.all(ids.map((id) => getUser(id)));
```

Say: “map each id to a promise of a User, wait for all of them, **users** is the array of Users.”

Plug in **ids = ["1", "2"]**: the **map** gives **[Promise<User>, Promise<User>]**. **Promise.all** waits for both and gives **Promise<User[]>**. **await** unwraps it: **users** holds **[{ id: "1", ... }, { id: "2", ... }]**, in input order.

```
const users = ids.map((id) => getUser(id));           // x users is Promise<User>[];
users[0].name is undefined
const users = await Promise.all(ids.map((id) => getUser(id))); // ✓ users is User[]
```

Review question: is there a `map` with an async callback that is not wrapped in `Promise.all`?

5. `forEach` with `await` inside waits for nothing

```
ids.forEach(async (id) => { await save(id) });  
console.log("done");
```

Say: “for each id, start an async callback that waits for save; then print done.”

Plug in `ids = ["1", "2"]`: `forEach` calls the callback twice, each returns a Promise, `forEach` throws both away and returns immediately. “done” prints before either save has finished. If a save fails, nobody catches it.

Review question: any `forEach` with `async` in it? Replace with `for...of` (sequential) or `Promise.all(ids.map(...))` (parallel).

6. `try` only catches what is awaited inside it

```
try { save(data); } catch (e) { log(e); }
```

Say: “try to call save; if it throws, log it.”

Plug in: `save` is async. It returns a Promise and the `try` block ends immediately, successfully. The Promise rejects later, outside the `try`. `log` never runs; the rejection is unhandled.

```
try { await save(data); } catch (e) { log(e); } // ✓ the rejection is thrown at the await,  
inside the try
```

Review question: every call inside a `try` that returns a Promise: is it awaited?

7. Read, wait, write is a race

```
const n = counts.get(k) ?? 0;  
await db.save(k);  
counts.set(k, n + 1);
```

Say: “read the count, wait for the save, write count plus one.”

Plug in two requests arriving together, `counts.get(k)` is 5: request A reads 5, waits. Request B reads 5, waits. A writes 6. B writes 6. Two increments, one lost.

Review question: is there shared state read before an `await` and written after?

Values, nulls, fallbacks

8. `undefined` is what you hold when nothing was put there

```
const o: { a?: number } = {};  
const v = o.a;
```

Say: “`o` is an object that may have a number `a`; `v` is set to `o.a`.”

Plug in: `o` is `{}`, no `a` inside. `v` holds `undefined`. No error. If the next line is `v.toFixed(2)`, that line throws `TypeError: Cannot read properties of undefined`.

Review question: for every `?` property, is it checked before `.something` is called on it?

9. `?.` stops at the first missing link and gives `undefined`

```
const city = user?.address?.city;
```

Say: “`user`, if it exists, dot `address`, if it exists, dot `city`.”

Plug in `user = { name: "Ann" }` (no address): `user` exists, `user.address` is `undefined`, the chain stops there, `city` holds `undefined`. Without `?.`, `user.address.city` throws.

Review question: is `undefined` an acceptable result here, or does the requirement say to error?

10. `??` fills in only a hole; `||` fills in any falsy

```
const size = Number(req.query.size) || 20;
```

Say: “`size` is the query’s size as a number, or else 20.”

Plug in `?size=0`: `Number("0")` is `0`. `0` is falsy. `||` replaces it. `size` holds `20`. The caller asked for 0 and got 20, silently. With `??`, `0` is not null/undefined, so `size` holds `0`.

Plug in `?size=abc`: `Number("abc")` is `NaN`. `NaN` is falsy. `||` gives `20`. `??` gives `NaN`, because `NaN` is not null/undefined. Neither operator validates; the fix is a real check.

Review question: can the left side legitimately be `0`, `""` or `false`? Then `||` is wrong. Is the left side validated at all?

11. `NaN` passes every comparison as `false`

```
const page = Number(req.query.page);
if (page < 1) return res.status(400).send("bad page");
```

Say: “`page` is the query’s page as a number; if it is less than 1, reject.”

Plug in `?page=abc`: `page` holds `NaN`. `NaN < 1` is `false`. The guard does not fire. `NaN` goes to the database.

Review question: every `Number(...)` from input: is there a `Number.isInteger` or `Number.isNaN` check, or only a range check?

12. `if (!x)` cannot tell “missing” from “zero”

```
if (!user.age) return "age required";
```

Say: “if `user.age` is falsy, reject.”

Plug in `age = 0`: `0` is falsy. A newborn is rejected as “age required”. The precise test is `user.age == null` or `=== undefined`.

Review question: can the value be a legitimate `0`, `""` or `false`?

Objects and arrays

13. `=` on an object copies the label, not the object

```
const copy = original;
copy.status = "paid";
```

Say: “`copy` is set to `original`; set `copy`’s status to paid.”

Plug in `original = { id: 1, status: "pending" }`: `copy` and `original` are two labels on ONE object. After line 2, `original.status` is also “paid”. Nothing was copied. To copy: `const copy = { ...original }`.

Review question: is an object that the caller still uses being modified through a second name?

14. `filter` returns a new array; the old one is untouched

```
orders.filter((o) => o.status === "paid");
const total = orders.length;
```

Say: “filter orders to the paid ones; `total` is the length of orders.”

Plug in 3 orders, 2 paid: line 1 builds a new array of 2 and throws it away (nothing on the left of it). `orders` still has 3. `total` holds 3, not 2.

```
const paid = orders.filter((o) => o.status === "paid"); // paid holds the 2
const total = paid.length; // 2 ✓
```

Review question: any `map`/`filter`/`slice`/`toUpperCase`/`trim` on its own line with nothing assigned?

15. `sort` changes the array in place AND returns it

```
const sorted = ids.sort();
```

Say: “sorted is set to ids sorted.”

Plug in `ids = [10, 9, 1]: .sort()` reorders `ids` itself to `[1, 10, 9]` (string order, because no comparator) and returns that same array. `sorted` and `ids` are one array. Two bugs in one line: the caller’s `ids` is reordered, and the order is wrong for numbers.

```
const sorted = [...ids].sort((a, b) => a - b); // copy first, then numeric comparator → [1, 9, 10]
```

Review question: `sort` on an array the caller still needs? `sort` on numbers without a comparator?

16. `map.get` on a missing key is `undefined`, then `.push` crashes

```
index.get(tag).push(doc);
```

Say: “get the list for this tag, push the doc onto it.”

Plug in a tag seen for the first time: `index.get(tag)` is `undefined`. `undefined.push` throws `TypeError`. The first call ever made to this code crashes.

```
(index.get(tag) ?? (index.set(tag, []), index.get(tag)!)).push(doc); // or the two-line if (!has) set form
```

Review question: every `map.get(k).something`: what if `k` is not there?

17. A key from a URL is a string; a key stored as a number will not match

```
const user = users.get(req.params.id);
```

Say: “`user` is the entry in `users` for the id from the URL.”

Plug in URL `/users/42`, map built with `users.set(42, ...)`: `req.params.id` is the string `"42"`. The Map key is the number `42`. Map keeps types, so `"42" !== 42`, `get` returns `undefined`, and the code reports “not found” for a user that exists.

Review question: do the key types on `set` and `get` match? Convert with `Number(...)` at the boundary.

18. `{ ...a, ...b }`: later keys win

```
const opts = { ...userOptions, ...DEFAULTS };
```

Say: “`opts` is a copy of `userOptions`, then `DEFAULTS` spread on top.”

Plug in `userOptions = { retries: 5 }`, `DEFAULTS = { retries: 3 }`: the second spread overwrites the first. `opts.retries` holds `3`. The user’s 5 was discarded. Defaults must come first: `{ ...DEFAULTS, ...userOptions }`.

Review question: in every spread merge, which side is supposed to win, and is it last?

19. `for...in` on an array gives you index strings, not elements

```
for (const i in items) total += items[i + 1];
```

Say: “for each key `i` in `items`, add the next item.”

Plug in `items = [10, 20]`: `i` is `"0"`, a string. `i + 1` is `"0" + 1 = "01"`. `items["01"]` is `undefined`. `total` becomes `NaN`.

Review question: `for...in` on an array? Should be `for...of` or a counting loop.

Functions and arrows

20. An arrow with braces and no `return` gives `undefined`

```
const doubled = nums.map(n => { n * 2 });
```

Say: “map each `n` to a block that computes `n` times 2.”

Plug in `nums = [1, 2]`: the block computes `2`, then `4`, and returns nothing from either. `doubled` holds `[undefined, undefined]`.

```
nums.map((n) => n * 2) // [2, 4]: no braces, the expression is returned
nums.map((n) => { return n * 2 }) // [2, 4]: braces with an explicit return
```

Review question: every `=> {` : is there a `return` inside?

21. A function declared to return `T` can still fall off the end

```
function find(xs: User[], id: string): User {
  for (const u of xs) if (u.id === id) return u;
}
```

Say: “`find` takes users and an id, returns a `User`; for each `u`, if the id matches, return it.”

Plug in an id that is not present: the loop finishes with no `return`. The function returns `undefined`. The caller, trusting `: User`, does `find(xs, "99").name` and crashes.

Review question: every path to the end of the function: does it `return` the declared type?

22. `=` inside a condition assigns and is always truthy

```
if (order.status = "paid") total += order.amount;
```

Say: “set `order.status` to `paid`; since `"paid"` is truthy, add the amount.”

Plug in a pending order: its status becomes `"paid"`, the amount is added. Every order is counted and every order is corrupted. `===` was intended.

Review question: any single `=` inside `if (...)`, `while (...)`, or a ternary?

23. The comparator must return a number, not a boolean

```
users.sort((a, b) => a.score > b.score);
```

Say: “sort users by whether `a`’s score is greater than `b`’s.”

Plug in two users: the arrow returns `true` or `false`. `sort` expects negative/zero/positive. `true` becomes `1`, `false` becomes `0`, and `0` means “keep order”, so pairs that should swap never do. Order is unreliable.

```
users.sort((a, b) => b.score - a.score); // descending; a number with a sign
```

Review question: every comparator: does it return a number?

24. A default parameter applies only when the argument is `undefined`

```
function page(size = 20) { ... }
page(null); page(0); page();
```

Say: “`size` defaults to 20 when not supplied.”

Plug in: `page()` → `size` is `20`. `page(0)` → `size` is `0` (0 was supplied). `page(null)` → `size` is `null`, not 20, because `null` is not `undefined`.

Review question: can a caller pass `null` where the author assumed the default would kick in?

Strings and numbers

25. String methods return new strings; the original is untouched

```
name.trim();
db.save(name);
```

Say: “trim name; save name.”

Plug in `name = " Ann "`: line 1 builds `"Ann"` and discards it. Line 2 saves `" Ann "` with the spaces. Fix: `name = name.trim()` (needs `let`) or `const clean = name.trim()`.

Review question: any string method on its own line?

26. Money in floats drifts

```
const cents = price * 100;
```

Say: “cents is the price times 100.”

Plug in `price = 19.99`: `19.99 * 100` is `1998.9999999999998`. Stored as an integer column, it truncates to `1998`. One cent lost per order. Fix: `Math.round(price * 100)`.

Review question: any arithmetic on money without rounding, or without integer cents from the start?

27. `Number("12px")` is `NaN`; `parseInt("12px")` is `12`

```
const width = Number(input);    const w2 = parseInt(input);
```

Say: “width is the input as a number; w2 is the input parsed as an integer.”

Plug in `input = "12px"`: width holds `NaN` (whole string must be numeric). w2 holds `12` (parses leading digits, stops at `p`). Neither throws. Python’s `int("12px")` would raise.

Review question: which conversion is used, and is its result checked?

Express handlers

28. `res.json()` sends but does not stop the function

```
if (!token) res.status(401).json({ error: "unauthorized" });
const data = await load(token);
res.json(data);
```

Say: “if no token, send 401; then load with the token and send the data.”

Plug in no token: line 1 sends the 401. The function keeps going. `load(undefined)` runs. Line 3 tries to send a second response: `ERR_HTTP_HEADERS_SENT`. Fix: `return res.status(401).json(...)`.

Review question: every early `res.send/res.json/res.status(...).json`: is it preceded by `return`?

29. Headers and query values are strings, or arrays, or missing

```
const key = req.headers["x-api-key"] as string;
limits.get(key);
```

Say: “key is the header, which I claim is a string; look it up.”

Plug in a request with no header: `req.headers["x-api-key"]` is `undefined`. The `as string` changes nothing at runtime. `key` holds `undefined`. `limits.get(undefined)` gives every keyless caller the same bucket. The cast hid the case.

Review question: every `as` on request data: what is the value really, when the field is absent or repeated?

30. `fetch` resolves on a 404; only `res.ok` tells you

```
const res = await fetch(url);
const user = await res.json();
```

Say: “wait for the fetch; wait for the body parsed as JSON; that is the user.”

Plug in a 404: `fetch` resolves (it only rejects on network failure). `res.status` is 404, `res.ok` is false. `res.json()` parses the error body, something like `{ error: "not found" }`. `user` holds that error object, and the code carries on as if it were a user.

```
if (!res.ok) throw new Error(`HTTP ${res.status}`); // before res.json()
```

Review question: is `res.ok` (or `res.status`) checked before the body is used?